Individual Report

MiniOS Alpha | Operating Systems Final Project

Student Name:

Group: Marquez and Friends

Assigned Role: Project Leader + Command-Line Interface

What did I contribute to the project?

I planned the project, set up the folder structure, and decided how the modules would talk to each other. I built the command shell in main.py that every other module plugs into, and I put the finished pieces together into one working program. I also wrote the README and kept the group on track.

What features did I personally develop?

- The MiniOS prompt that shows the current directory, for example MiniOS \Documents>
- The command loop and a dictionary that sends each command name to its handler
- Input parsing with shlex, so names with spaces in quotes work
- The start-up banner, the login step and the shutdown messages
- The 'Unknown command' response and a safety net that keeps the shell alive after unexpected errors
- Argument checks with Usage lines for commands that need input
- General commands: help, clear, date, time, echo, history, version, exit
- The README with run steps, accounts and the command list

What problems did I encounter?

- Names with spaces were split into separate words.
- A bug in any command could close the whole program.
- The clear command is different on Windows and other systems.
- The modules needed to work together (a process needs memory) without depending on each other too much.

How did I solve those problems?

- I used shlex.split for parsing and print a clear message when quotes are not closed.
- I wrapped every command in a try/except so an unexpected error shows a message and the prompt returns.
- I check the operating system and run cls on Windows or clear elsewhere.

- I agreed on a small interface with the group: the ProcessManager is given the MemoryManager when it is created, and only the shell talks to the user.

What did I learn from this project?

I learned how a command interpreter works: reading input, splitting it into a command and arguments, finding the handler, and reporting errors clearly. I also learned that a good split into modules makes teamwork easier, because each person can work on one file and the shell only needs to know each module's public methods.

Evidence of my contribution:

- main.py (MiniOS class, command dictionary, help text, run loop)
- README.md and requirements.txt
- Project folder structure and module interfaces

Sample output from my part of the program:

```
MiniOS \> hellooo
Unknown command.

Type 'help' to see the available commands.

MiniOS \> mkdir
Error: Missing argument.
Usage: mkdir <name>
```

Screenshots (attach in the screenshots folder and list the file names here):